ИНТЕРНЕТ-ТЕХНОЛОГИИ

npm

Менеджер пакетов для JavaScript и Node.js

Поляков Вячеслав 5040102/60201

Содержание

1. Что такое npm и зачем он нужен
2. package.json: паспорт проекта
3. Основные команды и npm scripts
4. Зависимости и версии (semver)
5. package-lock.json и node_modules
6. Безопасность
7. Демонстрация
8. Итоги

Что такое npm

Реестр пакетов

Огромная публичная база готовых библиотек на npmjs.com

Утилита командной строки

Команда **npm**: ставит, обновляет и удаляет пакеты, запускает скрипты

Часть Node.js

Устанавливается вместе с Node.js

npm – Node Package Manager. Создан Айзеком Шлютером, первый релиз в 2010 году. С 2020 года принадлежит GitHub.

Зачем нужен менеджер пакетов

Без npm

- Скачивать библиотеки вручную
- Копировать файлы в проект
- Самому следить за версиями
- Искать зависимости самих библиотек

С npm

- Одна команда: npm install
- Зависимости подтягиваются сами
- Версии зафиксированы в файлах
- Проект одинаково собирается у всех

Основные команды

Команда	Что делает
npm init	создаёт package.json
npm install	ставит все зависимости проекта
npm install имя	добавляет пакет в проект
npm uninstall имя	удаляет пакет
npm update	обновляет пакеты в допустимых версиях
npm run имя	запускает скрипт из package.json

```
"scripts": { "start": "node index.js" }  →  npm start
```

npm scripts: короткие имена для команд проекта. start и test запускаются без run, остальные – через npm run имя

package.json: паспорт проекта

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

name, version – имя и версия пакета

scripts – свои команды проекта

dependencies – что нужно для работы

Создаётся командой **npm init**

Два вида зависимостей

dependencies

Нужны программе во время работы

```
npm install express
```

Примеры: express, react, axios

devDependencies

Нужны только при разработке

```
npm install -D eslint
```

Примеры: eslint, jest, webpack

Версии: semver

`MAJOR.MINOR.PATCH  =  4.19.2`

- **MAJOR** – несовместимые изменения
- **MINOR** – новые возможности, всё старое работает
- **PATCH** – исправления ошибок

Запись	Разрешённые версии
^4.19.2	от 4.19.2 до 5.0.0 (не включая)
~4.19.2	от 4.19.2 до 4.20.0 (не включая)
4.19.2	только эта версия

package-lock.json и node_modules

package-lock.json

- Точные версии всех пакетов, включая вложенные
- Делает установку воспроизводимой
- Хранится в git

node_modules

- Папка со скачанными пакетами
- Может весить сотни мегабайт
- В git не хранится (.gitignore)

Команда **npm ci** ставит зависимости строго по lock-файлу.

Безопасность

Риски

- Тысячи вложенных зависимостей
- Уязвимости в пакетах
- Подмена похожим именем (typosquatting)
- Заражённые обновления

Что делать

- npm audit – проверка уязвимостей
- Фиксировать версии в lock-файле
- Проверять название и автора пакета
- Не тянуть лишние зависимости

История: в 2016 году из реестра удалили крошечный пакет left-pad, и сломались сборки множества проектов.

Демонстрация

```
$ mkdir demo && cd demo
$ npm init -y
$ npm install lodash
$ node index.js
$ npm audit
```

Смотрим, как появились package.json, package-lock.json и node_modules.

Итоги

- npm – реестр, утилита и часть экосистемы Node.js
- package.json описывает проект, lock-файл фиксирует версии
- Версии задаются по semver: ^ и ~ ограничивают обновления
- npm scripts упрощают повседневную работу
- Чужой код нужно проверять: npm audit и lock-файл